

"Softwaretechnik"
OCL
Formale Spezifikation von
Einschränkungen für Modellelemente

AG Software Engineering
Freie Universität Berlin, Institut für Informatik
<http://www.inf.fu-berlin.de/inst/ag-se/>

- Die OCL dient dazu, **Bedingungen für Methoden und Klassen** zu definieren.
 - formale Spezifikation von Einschränkungen für einzelne Modellelemente (z.B. Attribute, Operationen, Klassen)
 - OCL ist **deklarativ**, nicht prozedural, daher kann OCL nicht zur Beschreibung des Kontrollflusses verwendet werden.
- Verweise auf Abschnitte der OCL-Spezifikation in Version 2.4 zum genauen Nachlesen:
 - Beispiel: 7.5.14
 - <http://www.omg.org/spec/OCL/2.4/PDF/>
- Die folgenden Folien setzen OCL-Kenntnis aus der SWT-Vorlesung voraus

- Diese Constraints gelten für Membervariablen der Klasse, und zwar zu jeder Zeit während Objekte dieser Klasse existieren.

7.3.3

- Ausnahme: nur *während* der Ausführung einer Operation dürfen die Bedingungen verletzt werden

12.6

context <Klasse> **inv:** <Bedingung>

- Hinter dem OCL-Schlüsselwort **context** folgt der Klasse-Name, für die die Bedingung gelten soll.
 - Bei Invarianten bildet die relevante Klasse den Kontext des OCL-Ausdrucks. Das Schlüsselwort **self** kennzeichnet dann alle Exemplare der Klasse.
 - Ist der Kontext eindeutig, so kann das Schlüsselwort **self** weggelassen werden.

7.3.1

7.3.3

- Bei den Vor- und Nachbedingungen bilden Operationen den Kontext des OCL-Ausdrucks, wobei Operationen durch den Klassennamen gefolgt von `::` und der Methoden-Signatur bezeichnet werden.
- Hier kann pro Methode angegeben werden
 - welche Rahmenbedingungen beim Aufruf der Methode erfüllt sein müssen (Vorbedingung) und
 - wie der Rückgabewert oder der Zustand der Klasse zum Methoden-Ende aussieht (Nachbedingung).

7.3.4

```
context <Klasse>::<Operation>(<Parameter> : <Typ>): <ReturnTyp>  
  pre: <Bedingung 1>  
  pre: <Bedingung 2>  
  post: <Bedingung 3>
```

- Innerhalb einer Nachbedingung (post) greift man über @pre auf den Wert einer Variablen vor dem Aufruf zu. 7.5.14
 - Bsp.: Prüfungsverwaltungssystem: Ob ein Student bestanden hat, darf nur geändert werden, wenn er zugelassen ist. Der sonstige Zustand darf sich nicht ändern

```
context Student::setBestanden(b: bool)
  pre: zugelassen
```

```
context Student::setBestanden(b: bool)
  post: bestanden = b and
        zugelassen = zugelassen@pre
```

Student
zugelassen : boolean bestanden : boolean
setBestanden(b : boolean)

- Logische Operatoren sind: **and**, **or**, **xor**, **implies**, **not** 7.4 11.5.4
 - Bedeutung **implies**: Wenn die Bedingung vor dem Schlüsselwort gilt, dann gilt implizit auch die zweite Bedingung.

```
Student.allInstances()->forAll( p1, p2 |  
    p1 <> p2 implies p1.matrikel <> p2.matrikel )
```

Student
matrikel : String

<u>alice</u> : Student
matrikel = "0152"

<u>bob</u> : Student
matrikel = "0124"

Operation auf primitiven Typen

- Standard-Rechenoperatoren: +, -, * und /
- Relationale Operatoren: =, >, <, >=, <=, <>

11.3.1

11.5.1

11.5.2

11.5.5

```
score > 88.33
```

```
value <= next.value
```

```
self.questions->size() < 10
```

- String-Operationen: u.a.
+ (Konkatenation), **size** (Länge)

11.5.3

- Bei der Navigation zu anderen Klassen kann es vorkommen, dass man im Klassendiagramm über Assoziationen mit der Multiplizität "viele" navigiert.
 - In den zugehörigen Objektdiagrammen erhält man dann (potenziell) mehrere Objekte.
 - Um mit dieser Art von Situation umgehen zu können, bietet OCL einen zusätzlichen Typ *Sammlung* (Collection) an
- Zugriffe unterscheiden sich
 - auf Membervariable/Operation eines Objekts mit "."
 - auf Operationen einer Collection: "->"
- Beide Navigationsoperatoren unterstützen implizite Umwandlung (implizites collect und oclAsSet):
 - `aSet.name` $\Rightarrow$ `aSet->collect(name)`
 - `anObject->union(aSet)` $\Rightarrow$ `anObject.oclAsSet()->union(aSet)`

7.4.10

- Die meisten Assoziationen eines UML-Modells führen zu Collections, da oft eine **0..***-Beziehung besteht.
- OCL bietet vier Arten von Collections, die allesamt vom Typ "Collection" abgeleitet sind 7.5.11
 - **Set**: Kein Objekt kann doppelt vorkommen:
Set { 2, 5, 1, 88 }
 - **OrderedSet**: Wie Set, allerdings sortiert:
OrderedSet { 1, 2, 5, 88 }
 - **Bag**: Ein Objekt kann mehrfach vorkommen:
Bag { 1, 3, 4, 3, 5 }
 - **Sequence**: Wie Bag, allerdings sortiert:
Sequence { 1, 2, 3, 3, 45 }
- Die konkrete Art der Collection hängt von den UML-Eigenschaften des Assoziationsendes ab (unique, ordered, ...)

- **spieler** sei eine Collection

7.6

- `spieler->asSet()`: konvertiert die Elemente der Sammlung in eine Menge (d.h. unsortiert und ohne Dopplungen)
- `spieler->size()`: gibt die Anzahl der Elemente der Sammlung zurück
- `spieler->includes(object)`: gibt true zurück, wenn das object in der Sammlung ist
- `spieler->select(expression)`: wählt Elemente aus einer Sammlung aus, für die der expression-Ausdruck true ist
- `spieler->union(collection)`: vereinigt die Sammlung collection und gibt die Vereinigung beider Sammlungen zurück
- `spieler->intersection(collection)`: bildet die Schnittmenge aus der Sammlung und collection

context `Liga::getAktiveSpieler`

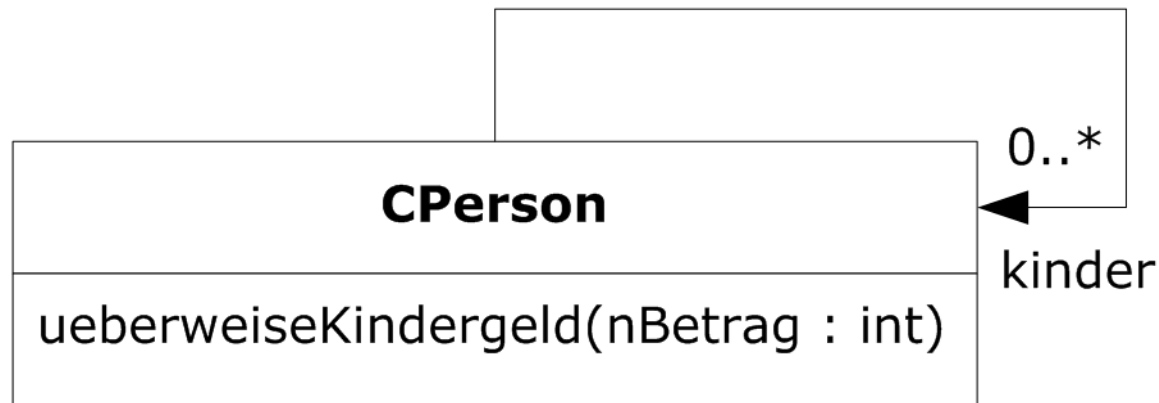
post: `result = turniere.spieler->asSet()`

- **Klasse.allInstances()** ist die Menge, die alle Objekte der Klasse enthält. 7.5.10
 - Bsp: "Zwei Studenten dürfen nie die gleiche Matrikelnummer haben."
`context Student inv: Student.allInstances()->forAll(s | s <> self) implies (s.matrikel <> self.matrikel)`
- Achtung: **allInstances()** nicht unnötig verwenden:
 - Bsp: "Eine Matrikelnummer ist positiv und mindestens 6-stellig."
`context Student inv: matrikelnummer >= 100000`
 - ...und nicht:
`context Student inv: Student.allInstances()->forAll(s | s.matrikelnummer >= 100000)`
- **allInstances()** also nur verwenden, wenn man wirklich über die Menge aller Exemplare der Klassen iterieren muss, um diese jeweils auf ein Prädikat zu überprüfen.

- Zwei OCL-Operationen ermöglichen es, über Sammlungen zu iterieren und dabei Prädikate auf jedem Element der Sammlung zu testen:
 - **forAll**(v|expression-with-v): ist true, wenn der Ausdruck für alle Elemente der Sammlung true ist 7.6.3
 - **exists**(v|expression-with-v): ist true, wenn der Ausdruck für mindestens ein Element der Sammlung true ist. 7.6.4

- Die Person hat eine Collection von Kindern, "kinder" (Assoziation zur Klasse "CPerson").
Die Operation "ueberweiseKindergeld" setzt die Bedingung "Person hat Kinder" voraus.

```
context CPerson::ueberweiseKindergeld(nBetrag : int)
  pre KinderVorhanden: self.kinder->size() > 0
```



- "Hat eine Person einen Ehepartner, so ist dieser volljährig"
 - Im CPerson-Objekt steckt eine Membervariable "ehepartner" vom Typ CPerson. (Im Diagramm dargestellt als eine Assoziation mit der Rollenbezeichnung "ehepartner" am anderen Ende.)
 - Bei Enden mit Maximum 1 (d.h. "0..1" oder "1"):
Auswertung ist *immer* ein Objekt (sonst ist es ein Set)
 - Auswertung von "self.ehepartner" ergibt entweder den Wert null (vom Typ OclVoid) oder einen Wert vom Typ CPerson.
 - Man kann aber dennoch "->notEmpty()" aufrufen, weil dabei implizit ein leeres bzw. ein-elementiges Set erzeugt wird

7.5.3

11.2.3

```
context CPerson inv VolljaehrigeEhe:  
  self.ehepartner->notEmpty() implies  
    self.ehepartner.alter >= 18
```

